



نام درس: سیستم های توزیع شده
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



تمرین شماره ۲

Problem 1

(الف)

در این سؤال، یک سیستم توزیع شده شامل سه پردازنده P1، P2 و P3 در نظر گرفته شده که میان یکدیگر پیام‌هایی را رد و بدل می‌کنند. هر کدام از این پردازنده‌ها به صورت مستقل و بدون هماهنگی با سایر پردازنده‌ها اقدام به گرفتن checkpoint می‌کنند. اکنون فرض می‌کنیم که پردازنده P2 پس از رسیدن به checkpoint شماره C10 دچار خرابی شده است. هدف ما این است که بررسی کنیم آیا می‌توان با استفاده از checkpoint‌های موجود، سیستم را به یک حالت consistent بازگرداند یا خیر.

برای آنکه مجموعه‌ای از checkpoint‌ها بتوانند یک recovery line معتبر و پایدار تشکیل دهند، باید در آن‌ها هیچ پیام یتیمی (orphan message) وجود نداشته باشد. یعنی اگر پیامی در یک پردازنده دریافت (receive) شده است، باید در یکی از checkpoint‌های پردازنده ارسال‌کننده نیز ارسال (send) شده باشد. همچنین اگر پیامی ارسال شده است، باید پیش از آن در یکی از checkpoint‌های سایر پردازنده‌ها دریافت شده باشد. در غیر این صورت، وضعیت حاصل ناپایدار و ناسازگار خواهد بود.

در اولین گام، تلاش می‌کنیم از جدیدترین checkpoint استفاده کنیم تا کمترین میزان rollback صورت گیرد. به همین منظور، اگر P2 در C10، P1 در C8 و P3 در C9 باشند، با بررسی پیام‌های تبادله شده درمی‌یابیم که پیام‌های شماره ۱۲ و ۱۳ که از سوی P2 ارسال شده‌اند، هنوز در P1 و P3 دریافت نشده‌اند. در نتیجه این پیام‌ها orphan هستند و این ترکیب نمی‌تواند یک recovery line معتبر باشد.

در گام بعد، P1 را به C5 و P3 را به C7 بازگردانیم و P2 همچنان در C10 قرار دارد. اما در این وضعیت نیز پیام‌های ۹، ۱۰ و ۱۱ در P2 دریافت شده‌اند، در حالی که هنوز در P1 یا P3 ارسال نشده‌اند. بنابراین این پیام‌ها نیز به عنوان پیام‌های یتیم شناخته می‌شوند و این وضعیت هم ناسازگار است.

اکنون اگر P2 را یک مرحله به عقب، یعنی به C6 بازگردانیم، و P1 و P3 را به ترتیب در C5 و C7 نگه داریم، مشاهده می‌کنیم که تمام پیام‌هایی که تا آن نقطه میان پردازنده‌ها رد و بدل شده‌اند، به درستی هم ارسال و هم دریافت شده‌اند. هیچ پیام یتیمی در این ترکیب وجود ندارد و بنابراین می‌توان نتیجه گرفت که ترکیب (C5, C6, C7) یک recovery line پایدار و معتبر است. این ترکیب به دلیل نزدیکی به وضعیت جاری سیستم، بهترین گزینه برای بازیابی محسوب

می شود.

اگر بخواهیم حالت های پایدار دیگری را نیز بررسی کنیم، می توانیم P1 را به C1 و P2 و P3 را در C6 و C7 قرار دهیم. اما در این حالت، پیام هایی مانند ۴ و ۷ که از سوی P2 و P3 ارسال شده اند، در P1 دریافت نشده اند، چرا که C1 قبل از دریافت آن ها قرار دارد. بنابراین این حالت نیز ناسازگار است. حال اگر P2 و P3 را به C2 و C3 بازگردانیم، ترکیب (C1, C2, C3) حاصل می شود که یک recovery line معتبر اما بسیار عقب تر از ترکیب قبلی است و منجر به از دست رفتن حجم زیادی از اطلاعات و محاسبات انجام شده خواهد شد.

در نتیجه، دو recovery line معتبر داریم: یکی ترکیب (C5, C6, C7) که ایده آل است و rollback بسیار کمی دارد، و دیگری (C1, C2, C3) که با وجود معتبر بودن، هزینه باز یابی بالاتری دارد و اطلاعات بیشتری را از بین می برد.

(ب)

در بخش دوم سؤال، هدف این است که نشان دهیم چگونه می توان با اضافه کردن logging و replay به سیستم، روند باز یابی را بهینه تر و دقیق تر انجام داد. در سیستم هایی که فقط از checkpoint استفاده می کنند، ممکن است در صورت خرابی یک پردازش، تمام پردازش ها نیاز به rollback داشته باشند تا به یک وضعیت پایدار بازگردند. اما اگر هر پردازش، علاوه بر گرفتن checkpoint، پیام ها و رخ داده های مهم خود را در یک stable log ذخیره کند، در صورت خرابی، تنها کافی است همان پردازش به آخرین checkpoint خود بازگردد و سپس با استفاده از داده های موجود در log، وضعیت خود را با اجرای مجدد پیام ها (replay) بازسازی کند.

برای مثال، اگر P2 در C6 دچار خرابی شود و در log خود پیام های ۹ و ۱۱ را ثبت کرده باشد، می تواند بدون تأثیر روی P1 یا P3، خود را به وضعیت دقیق قبل از خرابی بازگرداند. این روش باعث می شود که سایر اجزای سیستم بدون وقفه به کار خود ادامه دهند، محاسبات تکراری حذف شوند و باز یابی با سرعت و دقت بیشتری انجام گیرد. در نتیجه، ترکیب checkpoint + logging می تواند یک راهکار بسیار کارآمد برای باز یابی در سیستم های توزیع شده باشد و نسبت به استفاده صرف از checkpoint مزایای قابل توجهی ارائه دهد.

Problem 2

بخش (الف)

برای تحلیل اینکه کدام یک از سناریوهای مربوط به انتخاب leader در term های مختلف با وضعیت فعلی log ها مطابقت دارند، لازم است به دو اصل کلیدی الگوریتم RAFT توجه کنیم. اصل نخست این است که یک سرور زمانی می تواند به عنوان leader انتخاب شود که رأی اکثریت سرورها (در این سیستم پنج عضوی، حداقل سه رأی) را کسب کند. اصل دوم آن است که هر سرور تنها در صورتی به یک کاندید رأی می دهد که log آن کاندید از log خودش به روز تر یا دست کم برابر باشد، از نظر طول و نیز مقدار term در آخرین entry. با توجه به این اصول، هر یک از پنج گزینه ی

مطرح شده را به تفصیل بررسی می کنیم:

□ گزینه اول: نخستین نکته ای که باید به آن توجه کرد این است که در log های داده شده، تنها S1 دارای ورودی ای از term 3 است. هیچ سرور دیگری چنین entry ای ندارد. طبق قواعد پروتکل RAFT، تنها leader یک term مجاز است اولین entry را در آن term در log خود درج کند. بنابراین، مشاهده ی ورودی term 3 تنها در S1 یک نشانه قوی و روشن است که S1 در term 3 واقعاً به عنوان leader انتخاب شده و حداقل توانسته یک entry را در آن term ثبت کند. این مقدمه ای لازم برای سنجش اعتبار ادامه ی سناریو است.

حال باید بررسی کنیم که آیا ادعای رأی دادن S3 و S4 به S1 در term 3، از نظر قواعد رأی گیری RAFT امکان پذیر است یا نه. در پروتکل RAFT، رأی دهی مبتنی بر مقایسه به روز بودن log ها است؛ به این معنا که هر رأی دهنده فقط در صورتی می تواند به یک کاندید رأی دهد که log او از log خودش عقب تر نباشد. این مقایسه ابتدا بر اساس شماره term آخرین entry انجام می شود و اگر برابر بود، index بالاتر تعیین کننده ی به روز بودن است.

با نگاه به S3 log می بینیم که این سرور تنها دارای یک entry از term 1 است. در مقابل، S1 دو entry از term 1 دارد و سپس یک entry از term 3 در index 3. بنابراین، S1 نه تنها log ی طولانی تر دارد بلکه term آخرین entry آن نیز بالاتر است. در نتیجه، از دید S3، S1 کاملاً به روز تر محسوب می شود و S3 به راحتی می تواند به آن رأی دهد، مشروط بر اینکه هنوز در آن term به کاندیدای دیگری رأی نداده باشد.

در مورد S4 نیز همین تحلیل صادق است. S4 دارای دو entry از term 1 و یک entry از term 2 است. بنابراین آخرین entry آن در index 3 با term 2 قرار دارد. اما S1 در index 3 دارای entry از term 3 است. با مقایسه ی term ها درمی یابیم که term 3 در S1 بالاتر از term 2 در S4 است، بنابراین S1 log از S4 دید نیز به روز تر است و امکان رأی دادن S4 به S1 از نظر قوانین RAFT کاملاً وجود دارد. لازم به ذکر است که گاهی این رأی دادن به طور مستقیم و بلافاصله بعد از crash یا restart رخ می دهد، زیرا رأی دهنده ممکن است هنوز اطلاعاتی از سایر کاندیداها نداشته باشد.

برای درک بهتر اینکه این ترکیب رأی گیری چگونه می تواند در عمل اتفاق بیفتد، می توان به سناریویی که در متن تمرین اشاره شده است رجوع کرد. طبق آن سناریو، در term 1، سرورهایی مانند S1، S2، S4 یا S5 ممکن است به عنوان leader انتخاب شده باشند، اما S3 به دلیل نداشتن همه ی entry های term 1 نمی تواند leader شود و احتمالاً در میانه ی term 1 crash کرده است. سپس در term 2، S2 با دریافت رأی از S3، S4 و S5 leader می شود و ورودی هایی از term 2 را در log سرورهای مختلف ثبت می کند. اما پس از crash شدن مجدد S2 و قبل از اینکه بتواند entry های term 2 را به همه سرورها برساند، سرور S1 مجدداً کاندیدا می شود. چون log آن نسبت به S3 و S4 به روز تر است یا با آن ها برابر است، موفق می شود رأی آن ها را دریافت کند. پس

از دریافت سه رأی (از S1، S3 و S4)، S1 به leader تبدیل می شود و اولین entry از term 3 را ثبت می کند. در این شرایط، حتی اگر entry مربوط به term 3 در دیگر سرورها replicate نشده باشد، وجود آن در S1 log با چنین سناریویی سازگار است. و از آنجا که در RAFT فقط لازم است اکثریت آرا کسب شود (یعنی در این سیستم پنج عضو، حداقل سه رأی)، کسب رأی از S1، S3 و S4 برای leader شدن S1 کافی است. در نتیجه، گزینه ی اول نه تنها از نظر ساختار logها بلکه از نظر منطق رأی گیری و حتی با سناریوی عملیاتی مطرح شده در متن تمرین، کاملاً معتبر و قابل تحقق است. این گزینه برخلاف برخی تحلیل های سطحی که ممکن است آن را رد کنند، با بررسی دقیق تر و با لحاظ شرایط عملی مانند crash، recovery و زمان بندی صحیح رأی گیری، کاملاً محتمل است و می توان آن را یکی از سناریوهای صحیح در تحلیل وضعیت فعلی سیستم دانست.

□ گزینه دوم: «S1 leader در term 3 بوده و S1، S2 و S4 به او رأی داده اند.» این گزینه نیز از نظر الگوریتم RAFT قابل قبول نیست. در اینجا ادعا شده که S2 و S4 به S1 رأی داده اند. اما دارای ورودی هایی در term 2 و حتی term 4 است و log به مراتب کامل تر از S1 دارد. بنابراین به دلیل عقب بودن S1 log، S2 به او رأی نخواهد داد. همین موضوع در مورد S4 نیز صادق است، چراکه آن هم دارای ورودی در term 2 است. در RAFT، رأی به کاندیدی با log عقب تر ممنوع است. پس در مجموع این ترکیب رأی گیری غیرممکن است.

□ گزینه سوم: «S3 leader در term 1 بوده است.» این سناریو نیز با منطق RAFT همخوانی ندارد. در term 1، برای اینکه S3 بتواند leader باشد، باید log کامل آن term را در اختیار داشته باشد. اما S3 تنها یک entry از term 1 دارد، در حالی که سایر سرورها (مانند S1، S2، S4 و S5) حداقل دو ورودی از term 1 دارند. بنابراین S3 log ناقص تری دارد و نمی تواند leader معتبر آن term باشد، زیرا رأی سرورهایی با log کامل تر را جلب نخواهد کرد.

□ گزینه چهارم: «S2 leader در term 2 بوده و S3، S4 و S5 به او رأی داده اند.» این سناریو کاملاً محتمل است. در لحظه ای که S2 کاندیدا می شود، فرض می کنیم سرورهای دیگر یا تازه reboot شده اند یا logهایی مشابه یا کوتاه تر دارند. S3 دارای logی با تنها یک یا دو ورودی است و نسبت به S2 عقب تر است. S4 و S5 نیز دارای logهایی با term 1 هستند و term 2 را ندارند. بنابراین از نظر RAFT، S2 log از نظر طول و term از سایر سرورها بروزتر است و می تواند رأی آن ها را جلب کند. با کسب رأی از S3، S4، S5 و رأی خودش، S2 با اکثریت (چهار رأی) به عنوان leader term 2 انتخاب می شود.

□ گزینه پنجم: «S5 leader در term 2 بوده و S2، S4 و S5 ورودی term 2 را قبل از آغاز term 3 دریافت کرده اند.» این گزینه از چند جهت با وضعیت logها در تضاد است. اگر S5 leader term 2 بوده و موفق شده

ورودی 2 term را به S2 و S4 برساند، آنگاه باید در S4 log نیز ورودی مربوط به 2 term مشاهده می‌شد. اما در جدول log ارائه شده، تنها S2 دارای چنین entry است و S4 اصلاً 2 term را ندارد. بنابراین این فرض با شواهد ناسازگار است. از سوی دیگر، اگر واقعاً S5 چنین توزیعی انجام داده بود، آنگاه S1 دیگر نمی‌توانست در 3 term رأی کسب کند، چراکه log نسبت به سایر سرورها عقب‌تر بود. پس اساساً ورود S1 به 3 term و داشتن entry با 3 term قابل توضیح نبود. بنابراین این سناریو نیز با واقعیت log و قواعد RAFT همخوانی ندارد.

با توجه به بررسی بالا، می‌توان گفت تنها سناریویی که با منطق و وضعیت فعلی log تطابق دارد، گزینه‌ی چهارم است که در آن S2 به عنوان 2 term leader با رأی اکثریت انتخاب شده است.

بخش (ب)

در الگوریتم RAFT، یک entry زمانی committed تلقی می‌شود که در log اکثریت سرورها (حداقل سه سرور) وجود داشته باشد و توسط leader معتبر در term مربوطه ثبت شده باشد.

□ گزینه (a): [1, 1, 3]. تنها S1 دارای entry از 3 term است و این entry در هیچ سرور دیگری مشاهده نمی‌شود. بنابراین نمی‌تواند در اکثریت وجود داشته باشد و committed محسوب نمی‌شود.

□ گزینه (b): [1, 1, 4]. این entry تنها در S2 دیده می‌شود و سایر سرورها فاقد آن هستند. پس این گزینه نیز نمی‌تواند committed باشد.

□ گزینه (c): [1, 1, 2]. این پیشوند در چهار سرور (S1، S2، S4، S5) دیده می‌شود. از آنجا که S2 در بخش قبل به عنوان 2 term leader پذیرفته شده بود، این entry می‌تواند به درستی committed باشند.

بنابراین تنها گزینه‌ی سوم می‌تواند به عنوان وضعیت committed معتبر در نظر گرفته شود.

Problem 3

الف) تغییر شرط commitIndex به نیاز به اجماع همه سرورها

در پروتکل اصلی Raft، یک leader زمانی می‌تواند مقدار commitIndex را به مقدار N افزایش دهد که این مقدار در لاگ اکثریت سرورها ثبت شده باشد (یعنی $N \leq \text{matchIndex}[i]$ برای اکثریت سرورها برقرار باشد). حال اگر این شرط تغییر کند و به جای اکثریت، نیاز به تأیید همه سرورها باشد، یکی از مهم‌ترین ویژگی‌های Raft یعنی تحمل خطا (fault tolerance) از بین خواهد رفت.

پروتکل Raft به گونه ای طراحی شده که تا زمانی که اکثریت سرورها در دسترس باشند و بتوانند با یکدیگر ارتباط برقرار کنند، سیستم همچنان قادر است به درخواست های کلاینت پاسخ دهد. اما اگر شرط مورد نظر در سؤال اعمال شود، یعنی تنها زمانی commit صورت گیرد که همه سرورها مقدار مورد نظر را ثبت کرده باشند، آنگاه حتی خرابی یا قطعی یک سرور باعث توقف کامل سیستم می شود. این وضعیت باعث از بین رفتن خاصیت availability می شود و در نتیجه، سیستم دچار اختلال در پیشرفت (liveness) خواهد شد. در چنین حالتی، با وجود در دسترس بودن اکثریت، سیستم در حالت انتظار باقی می ماند و قادر به انجام هیچ پردازشی نخواهد بود. این در تضاد مستقیم با یکی از اهداف اصلی Raft یعنی تحمل خرابی تعداد محدودی از سرورها است.

ب) افزایش فوری commitIndex توسط leader بدون انتظار برای پاسخ AppendEntries

در این حالت فرض شده که leader، بلافاصله پس از ثبت یک ورودی جدید در لاگ خود، مقدار commitIndex را افزایش داده و آن را اجرا کند، بدون اینکه منتظر تأیید یا پاسخ از سایر سرورها برای AppendEntries باشد و بدون در نظر گرفتن مقادیر matchIndex. چنین رفتاری یکی از اصول کلیدی Raft یعنی Leader Correctness را نقض می کند.

در سناریوی مورد نظر، ممکن است leader چندین ورودی متوالی را ثبت کرده و آن ها را در state machine اجرا کرده باشد، در حالی که هنوز هیچ کدام از این ورودی ها در اکثریت سرورها تکرار (replicate) نشده اند. حال اگر leader به هر دلیل از کار بیفتد (crash) و در term بعدی، leader جدیدی انتخاب شود که این ورودی ها را در log خود نداشته باشد، آنگاه به دلیل قواعد log replication، ورودی های leader قبلی حذف و با لاگ leader جدید جایگزین خواهند شد.

در نتیجه، فرمان هایی که قبلاً اجرا و حتی پاسخ آن ها به کاربر داده شده بود، از لاگ حذف می شوند و این موضوع باعث inconsistency بین وضعیت state machine و لاگ واقعی سیستم می شود. این سناریو به طور مستقیم منجر به اجرای نادرست (incorrect execution) در سیستم می شود و خاصیت safety پروتکل Raft را نقض می کند.

ج) سناریوی شکاف شبکه و رقابت میان S1 و دیگر سرورها

در این سناریو، پنج سرور Raft داریم. S1 در term 17 رهبر بوده و پس از مدتی، به همراه S2 در یک شکاف شبکه ای (network partition) از سایر سرورها جدا شده است. سرورهای باقی مانده (S3، S4 و S5) را به عنوان رهبر در term 18 انتخاب کرده و فرمان هایی را commit و اجرا می کنند. سپس S1 از کار می افتد و پس از راه اندازی مجدد، به همراه S2 در چندین انتخابات شرکت می کند، تا جایی که مقدار currentTerm در آن ها به ۵۰ می رسد. در این میان، S3 از کار می افتد و شبکه به طور کامل ترمیم می شود و اکنون S1، S2، S4 و S5 می توانند با یکدیگر ارتباط برقرار کنند.

حال این پرسش مطرح است که آیا S1 می تواند به عنوان leader در term 50 انتخاب شود یا خیر؟ پاسخ این است که خیر، S1 نمی تواند leader شود.

اگرچه S1 رأی خود را دارد و می تواند رأی S2 را نیز کسب کند (زیرا آن دو در شرایط یکسان و با لاگ های مشابه عمل می کنند)، اما برای انتخاب شدن به عنوان leader به حداقل سه رأی نیاز دارد. از آنجا که S4 و S5 در term 18 با S3 همراه بوده اند و فرمان هایی را commit کرده اند، لاگ آنها شامل ورودی هایی از term 18 است. اما لاگ S1 و S2 تنها تا term 17 به روز شده است و در طول term های ۱۸ تا ۵۰ تنها انتخابات تکراری رخ داده اند، بدون اینکه لاگی جدید به این دو اضافه شود.

طبق قوانین رأی گیری در Raft، هر سرور تنها در صورتی به یک candidate رأی می دهد که سه شرط برقرار باشد: اول، term کاندید بزرگ تر یا مساوی با currentTerm رأی دهنده باشد. دوم، رأی دهنده هنوز در آن term رأی نداده باشد (votedFor = null). و سوم و مهم تر، لاگ کاندید به اندازه لاگ رأی دهنده به روز یا بروز تر باشد.

در این سناریو، لاگ S4 و S5 از S1 عقب تر نیست بلکه بروز تر است؛ چراکه آنها ورودی هایی از term 18 دارند و S1 حداکثر دارای ورودی هایی از term 17 است. بنابراین شرط سوم برقرار نیست و S4 و S5 به S1 رأی نخواهند داد. از آنجا که S3 نیز از کار افتاده و رأیی نمی دهد، S1 با داشتن تنها دو رأی (S1 و S2) نمی تواند اکثریت را به دست آورد و بنابراین leader نخواهد شد.

Problem 4

یکی از مشکلات اصلی و معروف در پروتکل (2PC) 2-Phase Commit، مسئله ی blocking است. Blocking به این معناست که یکی از شرکت کنندگان پروتکل، در شرایطی قرار می گیرد که نمی تواند تصمیم بگیرد که آیا باید commit کند یا abort، و بنابراین ناچار است به حالت انتظار برود تا تصمیم نهایی را از هماهنگ کننده دریافت کند. در این حالت، بدون دخالت هماهنگ کننده یا یک مکانیزم بیرونی، شرکت کننده نمی تواند پیشروی کند و سیستم دچار اختلال در liveness می شود.

برای درک بهتر این مشکل، یک سناریوی ساده را در نظر می گیریم:

فرض کنید در یک تراکنش توزیع شده، یک coordinator و دو participant وجود دارند. فرآیند 2PC با ارسال پیام prepare از سمت coordinator به هر یک از participant ها آغاز می شود. فرض کنیم هر دو participant پس از دریافت پیام prepare، پاسخ vote-commit ارسال می کنند. طبق پروتکل، پس از ارسال رأی yes توسط هر participant، آنها باید وضعیت لاگ خود را به حالت «آماده برای commit» تغییر داده و قفل های مربوط به منابع درگیر در تراکنش را حفظ کنند. آنها سپس منتظر پیام نهایی commit یا abort از سمت coordinator باقی می مانند.

حال فرض کنید قبل از اینکه coordinator پیام global commit را ارسال کند، به طور ناگهانی دچار خرابی شده یا

از دسترس خارج شود. در این شرایط، هر دو participant در وضعیتی باقی می ماند که اطلاعی از تصمیم نهایی ندارند. آن ها نمی دانند که آیا باید تراکنش را commit کنند یا abort. از آنجا که بر اساس پروتکل 2PC، هیچ کدام از participant ها مجاز نیستند به تنهایی تصمیم گیری کنند، هر دو در وضعیت بلا تکلیف و blocking قرار می گیرند. این حالت، یکی از ضعف های ذاتی 2PC است. در واقع، نقطه تمرکز تصمیم گیری (یعنی coordinator) به یک single point of failure تبدیل می شود. خرابی آن، مانع پیشروی کل فرآیند می شود. در چنین شرایطی، تنها دو راه حل ممکن باقی می ماند: یا coordinator دوباره به سیستم بازگردد و پیام نهایی را ارسال کند، یا یک سیستم مدیریتی بیرونی مانند timeout + manual recovery یا استفاده از پروتکل های پیشرفته تر (مانند 3PC یا Paxos Commit) در نظر گرفته شود.

مراجع